MAZE AI: Comparing Breadth-First, Depth-First and A* Search with a Manhattan Heuristic and Dynamic Replanning on Random Grid Mazes

Vigneshwar M
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
Vigneshwar.m.2024.cse@rajalakshmi.edu.in

Vishnuharan R
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
Vishnuharan.r.2024.cse@rajalakshmi.edu.in

***Abstract*—**Maze solving is a standard testbed for artificial intelligence search, yet breadth-first search (BFS), depth-first search (DFS) and A* are often taught only as pseudocode. This paper presents MAZE AI, a system that generates random solvable grid mazes, runs the three algorithms on identical instances, uses a Manhattan-distance heuristic for A*, checks every returned path against wall cells, and re-runs A* when a new obstacle blocks the agent. The system is provided as a Flask web application with a REST interface and as a self-contained console program. In an evaluation on 600 random solvable mazes (200 each on 10×10, 15×15 and 20×20 grids), BFS and A* returned paths of identical length at every size, while A* expanded 26.6%, 39.2% and 45.7% fewer nodes than BFS. DFS expanded fewer nodes than BFS but returned paths 1.8 to 2.6 times longer than optimal. A replanning example shows the agent recovering from an obstacle placed on its route. Execution times were below 0.35 ms for all methods and did not separate the algorithms reliably at these sizes.

Index Terms*—A* search, breadth-first search, depth-first search, dynamic replanning, grid pathfinding, Manhattan heuristic.*

I. INTRODUCTION

Pathfinding in grid environments is a basic problem in artificial intelligence. Robots in warehouses, characters in games and autonomous agents in indoor maps all represent their surroundings as cells that are either free or blocked, and then search for a sequence of moves from a start cell to a goal cell [1]–[3]. Because the representation is simple, a maze is a convenient testbed for the ideas behind state-space search: states, actions, a goal test, path cost and the order in which states are expanded.

Textbooks [3] describe breadth-first search, depth-first search and A *search, but students often meet them as pseudocode with little chance to see how each explores the same maze. The three algorithms make different trade-offs. BFS returns a shortest path on an unweighted grid but expands many cells. DFS uses little memory and reaches deep cells quickly, but its path can be much longer than necessary. A* uses a heuristic estimate of the remaining distance to focus the search, and returns an optimal path when that estimate never overestimates the true distance.

This paper presents MAZE AI, a system that makes these trade-offs measurable. It generates random solvable mazes, runs the three algorithms on identical instances, validates each path against the wall cells, reports path length, nodes explored and execution time, and recomputes a route with A* when a new obstacle blocks the agent. It is available as a web application and as a console program.

The contributions are: (i) a formulation and implementation of BFS, DFS and A *with a shared node representation and a strict wall-collision rule; (ii) a replanning procedure that re-runs A* from the agent's current position when an obstacle lies on the remaining route; (iii) a benchmark on 600 random solvable mazes at three grid sizes; and (iv) an analysis of where each algorithm's advantages appear and where they do not. Section II reviews related work, Section III formulates the problem, Section IV describes the method, Section V the implementation, Section VI the experiments, and Section VII concludes.

II. RELATED WORK

A. Uninformed Search

Moore [1] studied shortest paths through mazes and showed that expanding cells in order of distance from the start finds the shortest route. This is the breadth-first strategy used in MAZE AI, implemented with a first-in-first-out queue. Tarjan [4] gave the linear-time depth-first formulation that underlies many graph algorithms. DFS keeps only the current branch on its stack but does not guarantee short paths.

B. Heuristic Search

Hart, Nilsson and Raphael [2] defined A* as best-first search over $f(n) = g(n) + h(n)$ and proved that it is optimal when the heuristic is admissible. Russell and Norvig [3] present the algorithm with the consistency condition. For four-direction grid movement, the Manhattan distance is both admissible and consistent, because each move changes exactly one coordinate by one unit.

C. Replanning

When the environment changes after a plan is made, a single search is not enough. D *Lite [5] reuses information from earlier searches to repair a path incrementally. MAZE AI instead recomputes the route with A** from the current position. This is less efficient on large grids, but it is simpler to explain and sufficient for the small environments studied here. Cormen et al. [6] give the standard analysis of the data structures used by these algorithms.

III. PROBLEM FORMULATION

A maze is an $N \times M$ grid of cells indexed by (r, c). Each cell is either free (value 0) or a wall (value 1). A move goes from a cell to one of its four neighbors and costs one unit. The start is $s = (0, 0)$ and the goal is $g = (N-1, M-1)$, and both are always

free. A *solution* is a sequence of free, pairwise-adjacent cells from s to g. Let L be the number of moves in a solution, E the number of nodes expanded during search, and T the execution time in milliseconds. An algorithm is *optimal* if it returns a solution with the smallest L among all solutions.

In the replanning setting, an agent follows a solution and occupies cell p_t after t moves. An obstacle is then placed on a free cell o of the remaining route. The agent must find a new solution from p_t to g that avoids every wall, including o.

IV. PROPOSED METHOD

A. Maze Generation

Each non-terminal cell is made a wall with probability 0.30 and a free cell otherwise. A random grid may have no route, so a BFS checks solvability. The generator retries up to 100 times. If every attempt fails, it returns an empty grid, so the application always has a valid maze. The same generator is used by the web application and by the benchmark.

B. Search Algorithms

All three algorithms share a node record that stores a position, a parent pointer, a path cost g and a heuristic value h, and a neighbor function that returns only in-bounds cells with value 0. Table I summarizes their differences.

TABLE I
CHARACTERISTICS OF THE IMPLEMENTED ALGORITHMS

Algorithm	Frontier	Shortest path	Heuristic
BFS	FIFO queue	Yes	None
DFS	LIFO stack	No	None
A*	Priority queue	Yes	Manhattan (admissible)

BFS marks a cell as discovered when it is enqueued, so each cell enters the queue at most once. Because every move has unit cost, the first time the goal is dequeued its path is a shortest path.

DFS keeps a stack. Neighbors are pushed in reverse order so that the preferred direction is explored first, and a cell is marked visited when it is popped. The returned path is valid but not necessarily shortest.

A *orders the frontier b*

$$f(n) = g(n) + h(n) \tag{1}$$

where g(n) is the number of moves from the start to n, and h(n) is the Manhattan distance to the goal,

$$h(n) = |r_n - r_g| + |c_n - c_g| \tag{2}$$

A* records the best known cost of each cell and enqueues a neighbor only when it reaches that cell more cheaply. The heuristic is admissible: each move changes one coordinate by one unit, so at least $|\Delta r| + |\Delta c|$ moves are needed, and walls can only lengthen a route. It is also consistent, since one move changes h by at most one, which equals the cost of the move. Under these conditions the first expansion of the goal yields an optimal path [2], [3].

C. Path Validation

Search moves only through free neighbors, so a returned path cannot contain walls by construction. The system repeats the check independently anyway. The validator collects the wall coordinates and tests each coordinate of the returned path against them, and a failing path is discarded. The check confirms wall avoidance but not adjacency, which follows from how the search builds paths.

D. Dynamic Replanning

The agent follows the initial A *route. When a dynamic obstacle is introduced, the system tests whether it lies on the remaining route. If it does, A* is run again from the current position and the agent continues on the new route. If no route exists, the agent reports that it is stuck. The command-line program performs this check during its simulation, and the web API exposes the same computation through an endpoint that accepts the current grid and position.

V. IMPLEMENTATION

The web application is written in Python with Flask [7]. The server provides three JSON endpoints. The first generates a maze. The second runs the selected algorithm and returns the path, the visited order, the number of nodes explored, the path length and the execution time. The third recomputes an A* route from the current position on an updated grid. The frontend uses HTML, Tailwind CSS and vanilla JavaScript to draw the grid, animate the search, and display the metrics. The search code uses only the Python standard library, namely `collections.deque` and `heapq` [8]. The companion program `maze_ai.py` contains its own copy of the three algorithms, a comparison table and a step-by-step replanning simulation with colored output.

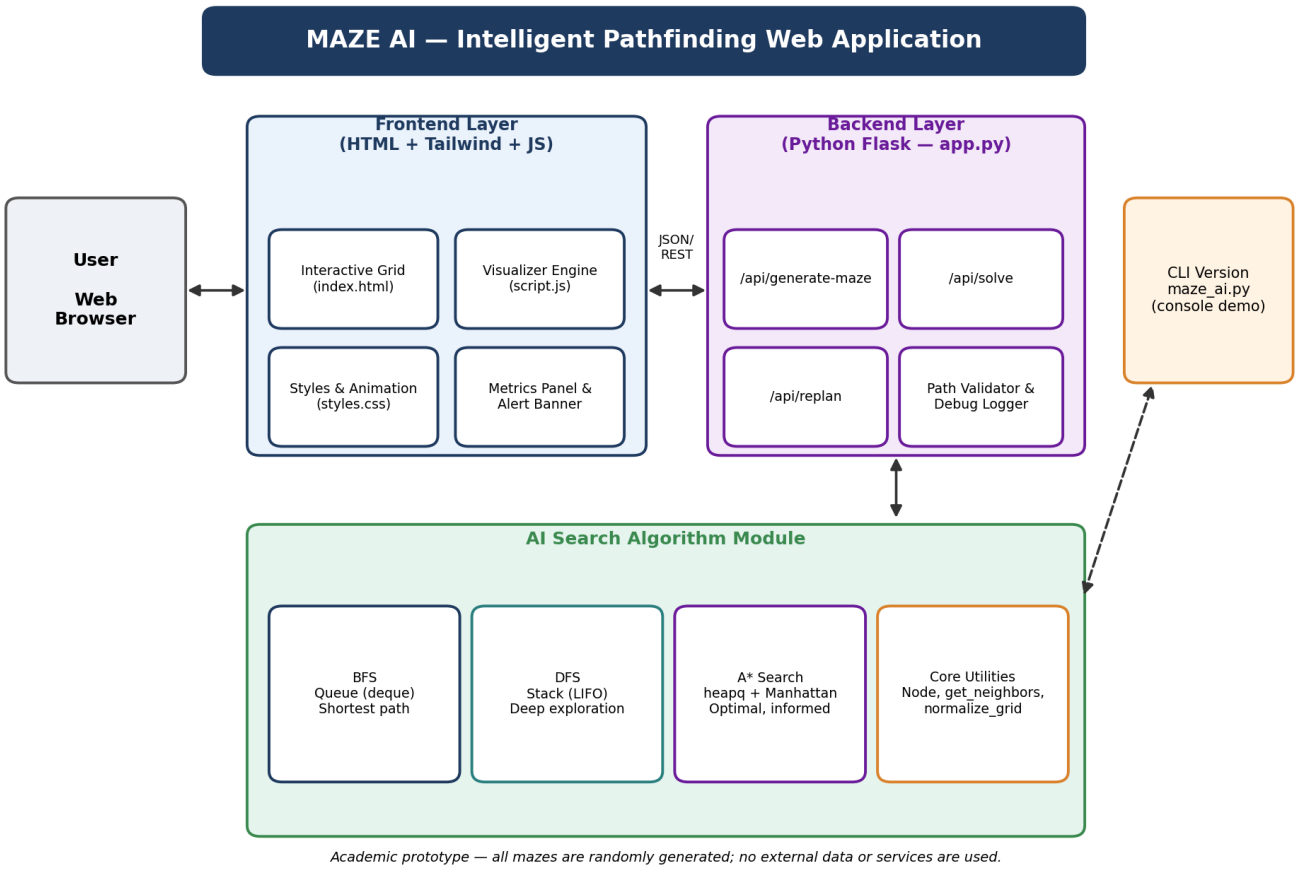


Fig. 1. System architecture of MAZE AI. The browser dashboard communicates with a Flask backend that holds the graph model, the search algorithms, the validator and the replanner.

VI. EXPERIMENTAL EVALUATION

A. Setup

Experiments use grids of 10 × 10, 15 × 15 and 20 × 20 cells. For each size, 200 solvable mazes were generated with wall probability 0.30 using random seeds 0 to 199, giving 600 mazes and 1800 searches in total. Each maze was solved by BFS, DFS and A* using the functions in `app.py`, and the metrics were averaged per size. Movement is four-directional with unit cost. Execution time was measured around each search with a high-resolution timer on a single development machine.

B. Case Study

Fig. 2 shows the three algorithms on one 10 × 10 maze. BFS and A *both return an 18-step route, but A* expands 38 cells against 63 for BFS. DFS expands 60 cells yet returns a 44-step route.

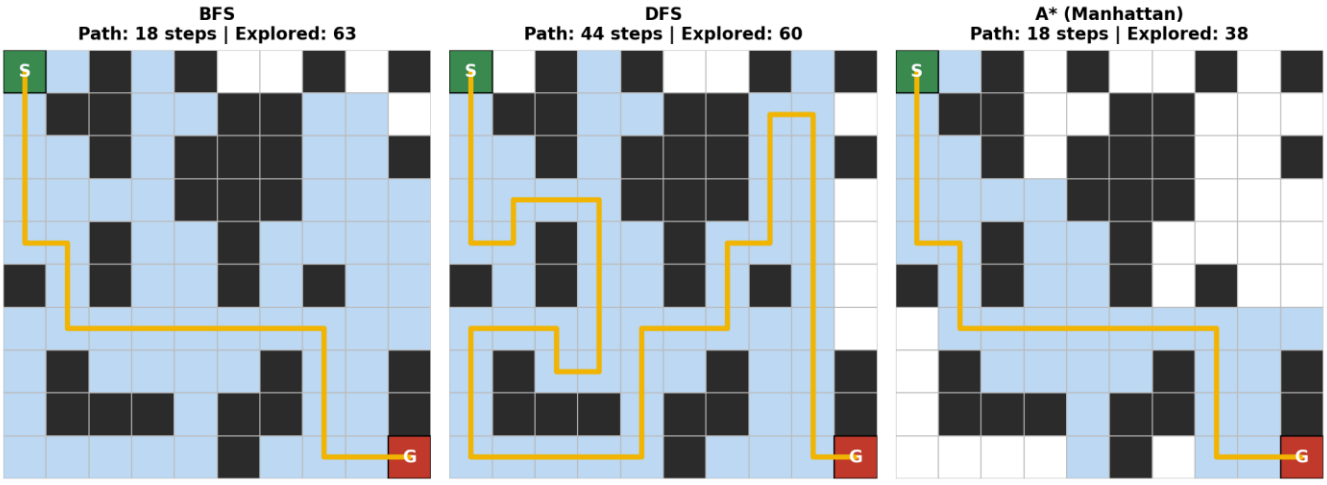


Fig. 2. Search behavior of BFS, DFS and A* on the same 10 × 10 maze. Light blue cells were expanded, the yellow line is the returned route, and S and G mark the start and goal.

C. Benchmark Results

Table II reports the averages. BFS and A *returned identical average path lengths at every size, which is consistent with both being optimal on this grid. A* expanded 26.6%, 39.2% and 45.7% fewer nodes than BFS at 10 × 10, 15 × 15 and 20 × 20, so its advantage grows with the grid. DFS expanded fewer nodes than BFS, but its routes were on average 1.83, 2.22 and 2.59 times longer than the optimal length, which is the price of its low memory use.

TABLE II
AVERAGE RESULTS OVER 200 RANDOM SOLVABLE MAZES PER GRID SIZE

Grid	Algorithm	Path length	Nodes expanded	Time (ms)
10×10	BFS	18.25	66.1	0.107
10×10	DFS	33.38	57.9	0.112
10×10	A*	18.25	48.5	0.128
15×15	BFS	28.56	146.6	0.219
15×15	DFS	63.33	123.6	0.222
15×15	A*	28.56	89.2	0.224
20×20	BFS	39.33	262.4	0.322
20×20	DFS	101.94	221.9	0.330
20×20	A*	39.33	142.5	0.295

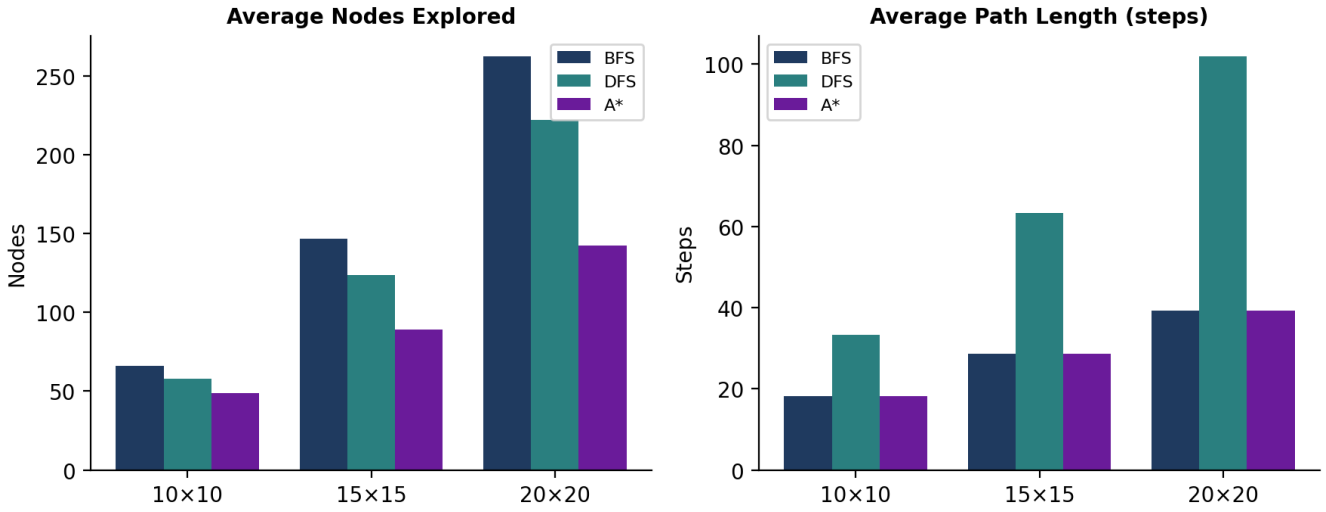


Fig. 3. Average nodes expanded (left) and average path length (right) by grid size and algorithm, over 200 mazes per size.

D. Execution Time

Average execution times ranged from 0.107 to 0.330 ms. On the 10 × 10 grid A *was slightly slower than BFS (0.128 ms against 0.107 ms) even though it expanded fewer nodes, plausibly because each heap operation costs more than a queue operation. On the 20 × 20 grid A* was faster (0.295 ms against 0.322 ms). These differences are small, come from one timing per search on one machine, and should not be used to rank the algorithms. Node counts are the more reliable comparison.

E. Replanning Example

Fig. 4 shows the replanning procedure on the 10 × 10 maze. The initial A *route has 18 moves. After two moves the agent is at (2, 0), and an obstacle is placed at (6, 3), which lies on the remaining route. A* from the agent's position finds a 16-move route after expanding 34 cells, so the total journey is 2 + 16 = 18 moves. The new route is the same length as the original one because an equally short detour existed.

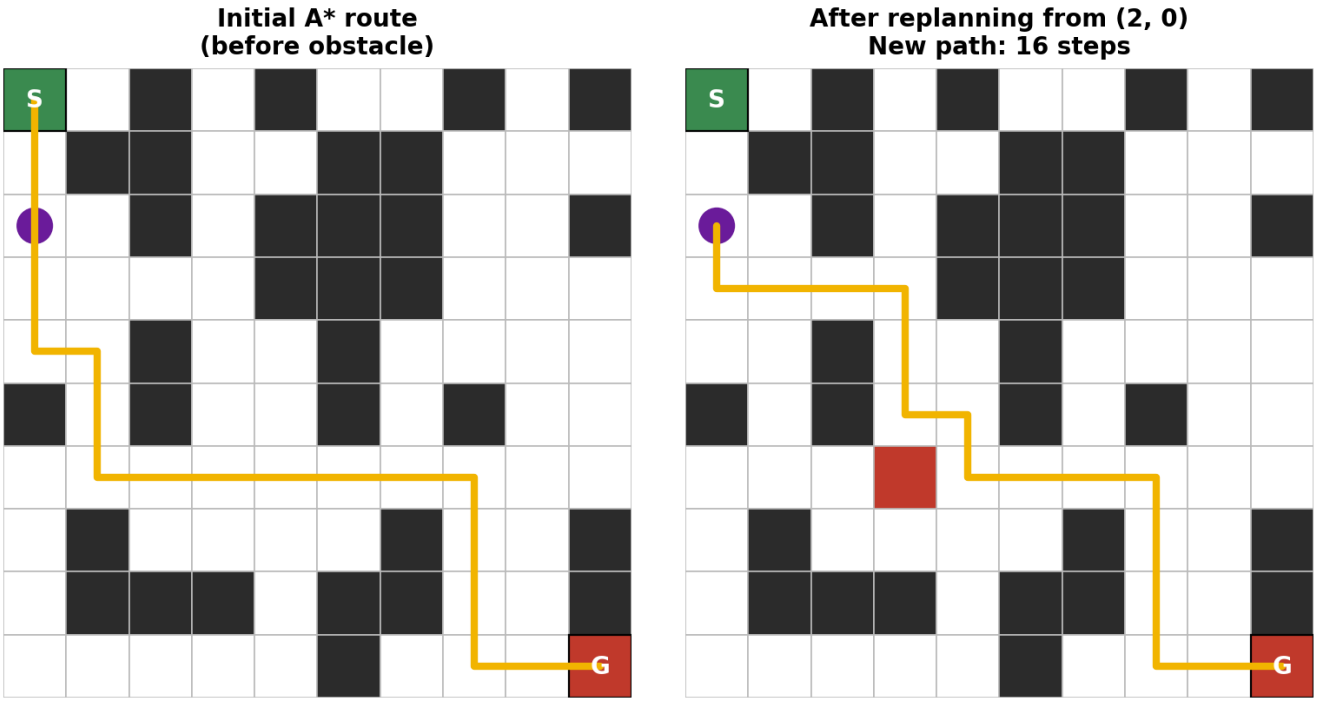


Fig. 4. Dynamic replanning with A. *Left: the initial route and the agent's position. Right: after an obstacle (red) is placed on the remaining route, A* computes a new route from the agent's current position.

F. Threats to Validity

The mazes are synthetic, four-connected and generated at a single wall density, so results may differ for perfect mazes, corridor-structured layouts or weighted terrain. The grids are small, and timing depends on the machine and on the implementation language. The validator checks wall membership only, and the replanning study covers one obstacle scenario.

VII. CONCLUSION AND FUTURE WORK

MAZE AI shows, on 600 random solvable mazes, that A *with the Manhattan heuristic returns optimal paths like BFS while expanding substantially fewer nodes, and that the saving increases with grid size. DFS expands fewer nodes than BFS but returns routes up to 2.6 times longer than optimal. Replanning by re-running A* from the agent's current position recovers from a new obstacle at the cost of one fresh search.

Future work includes weighted terrain with Dijkstra's algorithm and weighted A*, diagonal moves, moving obstacles, incremental replanning with D* Lite, evaluation on perfect mazes and real floor plans, repeated timing runs, and a single shared library so that the web and console versions use the same code.

REFERENCES

[1] E. F. Moore, "The shortest path through a maze," in *Proc. Int. Symp. Theory of Switching*, Cambridge, MA, USA: Harvard Univ. Press, 1959, pp. 285–292.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[3] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[4] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, pp. 146–160, 1972.

[5] S. Koenig and M. Likhachev, "D *Lite," in* Proc. 18th Nat. Conf. Artif. Intell. (AAAI)*, 2002, pp. 476–483.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[7] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[8] Python Software Foundation, "Python 3 documentation: collections and heapq." [Online]. Available: https://docs.python.org/3/